

# Working with AI

*...and owning every line*

Headless CMS Admin Panel · Agile Monkeys Frontend Challenge 2026

11 DEFECTS FOUND

237 TESTS

5 MILESTONES

"Please, use AI. We do too.  
**Just own every line you send us."**

1

### **The easy half**

Everyone will show that they used AI. Speed is table stakes now — it proves nothing on its own.

2

### **The half that counts**

Ownership means knowing what the AI got wrong — and being able to prove you caught it.

# A fast implementer, not a decision-maker

## I OWNED

Scope and priorities

Architecture and trade-offs

What counts as "verified"

Accepting or rejecting every diff

## THE AI OWNED

Turning a decided approach into code

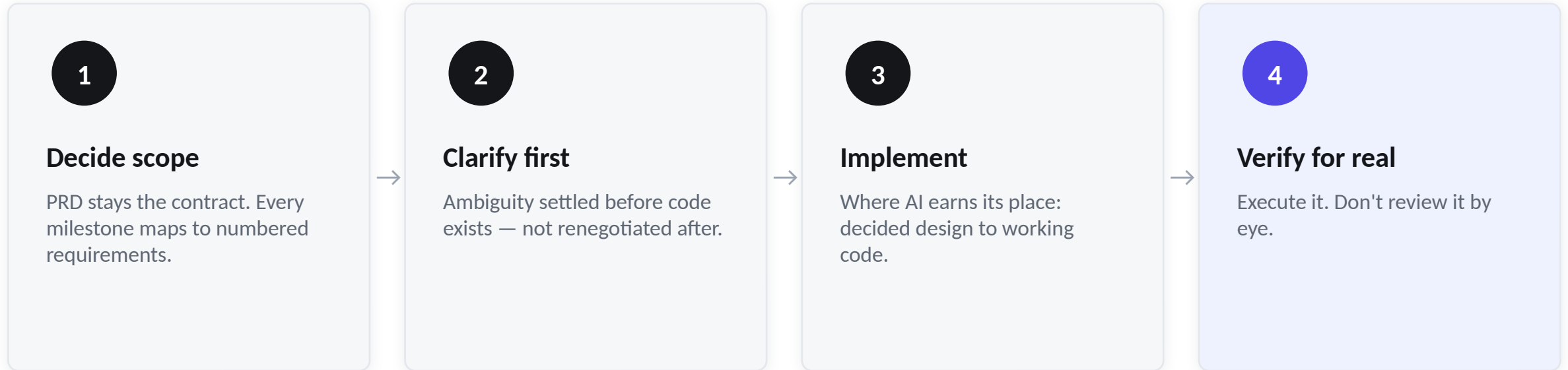
Boilerplate: types, forms, tables, SQL

Tests for the cases I named — then more

Documenting decisions once made

*The rule: no code merged because it looked right. Everything was tested, executed against a real dependency, or driven in a browser — or the docs say plainly that it wasn't.*

# Same cycle, every milestone



Then record it. Each milestone has a document covering what was built, what was decided — and how it was verified. Three strategy decisions were reversed mid-build; the docs say so.

## METHOD

# What "verified" had to mean

AI-written code is confidently wrong in a way that reads well. So I stopped reviewing by eye and started executing.



### Run the SQL against a real database

PostgreSQL 16 locally — not just read the migration



### Call the API over HTTP

curl against the running server, not only unit tests



### Drive the UI in a real browser

Headless Chromium: submit, search, sort, retype



### Simulate what you cannot reach

Built a stub websocket server speaking the Phoenix protocol



### Test the nasty cases first

Not coverage — the inputs where a plausible implementation is wrong

## THE RESULT

237

tests, 8 files

2,423

lines of test

8,267

lines of source

# Eleven defects — all in code that looked correct

3

## Edge semantics

Impossible dates, ambiguous conversions

2

## Failure paths

Leaked internals, wrong status code

2

## Second runs

Non-idempotent migration, silent type collapse

4

## Concurrency & the DOM

Overwrite-on-refresh, effect violations, a stale media listener, hover beating checked

*Every one of these passed review by eye. None would have been caught by reading the diff.*

# The validator that accepted 31 February

## WHAT AI WROTE

```
!Number.isNaN(Date.parse(value))
```

Reads perfectly. Date.parse does not fail on 2026-02-31 — it rolls over to 3 March and reports success.

## AFTER

Round-trip the parsed parts

Rebuild the date from year/month/day and check it survives. Leap years come out right for free.

## It happened twice

The same class of bug reappeared in the migration engine three milestones later: 2026-02-29 was silently becoming 2026-03-01 through a lenient fallback parse. In a feature whose entire purpose is to not corrupt data during a schema change.

# The reorder that would have rewritten every entry

```
JSON.stringify(a) !== JSON.stringify(b)
```

A reasonable-looking "did this row actually change?" check — used to skip writes for rows that did not move.

## It is key-order sensitive

The draft rebuilds each object in field order — so simply reordering fields made every row look changed.

## What that would have cost

- Every entry rewritten for a cosmetic change
- Every `updated_at` bumped — audit trail destroyed
- Every connected client woken by real-time

## Caught by

A test asserting that a purely cosmetic change writes nothing at all.

# Real-time silently killed conflict detection

1

## Milestone 2

The edit form sends the `updated_at` it loaded with. The server refuses the write if the row moved on.

2

## Milestone 4

Real-time calls `router.refresh()`, which re-renders the page with a NEW `updated_at`.

3

## The break

A token tracking that prop adopts a colleague's save as its own baseline.

**The next save overwrites their work — while reporting success.**

Conflict detection would still look like it worked, and would in fact be dead. Found by asking what the feature I just built does to the feature I built two milestones ago — a question about the system, not the code in front of me.

# Reliable at structure. Weak at edges.

STRONG

## Structure

- Well-organised modules
- Consistent naming and types
- Idiomatic framework usage
- Thorough-looking test suites

WEAK

## Edge semantics

- Impossible values
- Failure paths
- Running something a second time
- Two things happening at once

*It produces a well-organised date validator that accepts 31 February. The structure is never the problem.*

# Four times I threw the AI's answer away



## Dropped shadcn/ui

It was in my own plan. The surface actually needed was a few form controls — native `<select>` and `<dialog>` give accessibility for free, without a dependency tree.



## Dropped React Hook Form

For a form whose fields are values in an array, one state object plus the runtime schema is less machinery than a Controller per dynamic field.



## Reclassified date → number as lossy

AI called it safe: epoch millis are recoverable. True — but 1843-10-01 becoming -3986064000000 is not what an editor expects. The bytes survive; the meaning does not.



## Kept the transform out of SQL

The tempting route to atomic migrations. It would mean two implementations of the same rules — and any drift produces a preview that lies.

# What it bought, what it cost

## BOUGHT

### Breadth

Five milestones, a real-time layer, an HTTP API and an atomic migration engine in one working session. Writing the boilerplate by hand would have meant cutting scope — and the saved time went into schema evolution, the part the brief actually cares about.

## COST

### Verification is not free

Roughly a third of the effort went into proving things rather than writing them: a local Postgres, a websocket stub, a headless browser. That is the real price of working this fast. Skipping it produces something that demos well and breaks on contact with real data.

## WOULD CHANGE

### Ask it earlier

"What does this break that already works?" — I would run that pass at the start of each milestone rather than midway. The concurrency defect was found by luck of sequencing, not by process.

# I used AI to write most of the lines and none of the decisions.

It was fastest where the design was already settled, and least trustworthy at edges — empty values, failure paths, second runs, concurrency. So the process was built around executing things rather than reading them.

*The judgement calls stayed mine — and so does responsibility for every line in the repository.*